

ABSTRACT

The Blood Donor Finder is a web-based application developed to address the critical challenge of locating suitable blood donors during medical emergencies. By maintaining a centralized database of registered donors including their names, blood groups, contact information, and locations the system replaces the inefficient reliance on personal contacts with a streamlined, searchable platform. Built using a tech stack of HTML, CSS, and JavaScript for the front-end, Node.js for the backend, and My SQLite for data management, the application provides a user-friendly environment for both donor registration and rapid searching.

The primary objective of the project is to reduce life-threatening delays by ensuring quick access to vital information and creating broader awareness about blood donation. Operating on a local server and accessible via standard web browsers, the current system follows a clear workflow of data storage and donor matching. Looking forward, the project aims to expand its social impact through future enhancements such as real-time location tracking, SMS/Email notifications, and direct integration with hospital databases, ultimately serving as a robust tool for public health support.

The modern landscape of emergency response and community resource allocation often suffers from critical delays caused by fragmented data silos and non-responsive communication channels. This project addresses these inefficiencies by developing a high-performance, full-stack digital ecosystem designed to facilitate instantaneous connections between service providers and end-users. At its core, the system utilizes an asynchronous, event-driven Node.js backend combined with Web Sockets to enable bidirectional, low-latency communication, which is essential for real-time tracking and live status updates. The development process followed the Agile methodology, allowing for iterative testing of the search algorithms and donor/provider registration modules to optimize the user journey. Security was treated as a fundamental requirement, incorporating JSON Web Tokens for robust session management and following the AWS Shared Responsibility Model to protect sensitive user data against common vulnerabilities such as injection attacks and unauthorized access.

TABLE OF CONTENT

S.NO	CONTENTS	PAGE NO
1	INTRODUCTION	1-14
1.1	INTRODUCTION	2
1.2	INTRODUCTION , HISTORY, APPLICATION OF JAVASCRIPT	2 - 4
1.3	INTRODUCTION , HISTORY, APPLICATION OF CSS	5 - 6
1.4	INTRODUCTION , HISTORY, APPLICATION OF HTML	6 - 8
1.5	THEORY AND CONCEPTS	8 - 10
1.6	INTRODUCTION TO DATABASE MANAGEMENT SYSTEM	11
1.7	INTRODUCTION , HISTORY, APPLICATION OF DBMS	12 - 13
1.9	REQUIREMENT SPECIFICATION	14
2	SYSTEM STUDY	15-23
2.1	FEATURES OF THE SYSTEM	16 - 17
2.3	SOFTWARE DESCRIPTION	17 - 20
2.4	FUNCTION AND REQUIREMENT	21 - 22
2.5	NON-FUNCTIONAL REQUIREMENT	22 - 23

TABLE OF CONTENT

S.NO	CONTENTS	PAGE NO
3	SYSTEM DESIGN	24 -35
3.1	ER DIAGRAM	25
3.2	MAPPING FROM ER DIAGRAM TO SCHEMA DIAGRAM	25 – 27
3.3	NORMALIZATION	28 – 29
3.4	ASSUMPTIONS	30
3.5	SCHEMA DIAGRAM , FUNCTIONAL DEPENDENCY	30 - 31
3.6	INPUT DESIGN OF THE SYSTEM	32 - 34
3.7	OUTPUT DESIGN OF THE SYSTEM	34 - 35
4	METHODOLOGY	36 - 41
4.1	STAGE OF THE ITERATIVE DEVELOPMENT PROCESS	37 – 38
4.2	ADVANTAGES OF ITERATIVE BLOOD DONOR WEBSITE	39
4.3	DISADVANTAGES OF ITERATIVE BLOOD DONOR WEBSITE	40 - 41
5	TESTING	42 – 46
5.1	INTRODUCTION TO TESTING	43
5.2	TYPES OF TESTING	43 – 45
5.3	TESTING PERFORMED	45 - 46

TABLE OF CONTENT

S.NO	CONTENTS	PAGE NO
6	IMPLEMENTATION	47 – 53
6.1	TABLES USED IN DATABASE	48 – 50
6.2	DATABASE CONNECTION	51
6.3	INSERT OPERATION	51 – 52
6.4	UPDATE OPERATION	52
6.5	DELETE OPERATION	53
7	SNAPSHOTS	54 – 58
7.1	LOGIN PAGE	55
7.2	HOME PAGE	56
7.3	REGISTRATION PAGE	56
7.4	FIND DONOR PAGE	57
7.5	ADMIN PAGE	57
7.6	ABOUT PAGE	58
8	CONCLUSION	59 – 60
8.1	CONCLUSION	60
8.2	FUTURE WORKS	60
9	REFERENCE	61 – 62
9.1	BOOKS	62
9.2	WEBSITE	62
9.3	SCANNER	63

LIST OF FIGURES

S.NO	CONTENTS	PAGE NO
3.1	ER DIAGRAM FOR BLOOD DONOR FINDER	25
3.2	MAPPING OF REGULAR ENTITIES	26
3.3	MAPPING BINARY 1: N RELATIONSHIP	27
3.4	MAPPING OF BINARY M: N RELATIONSHIPS	27
3.5	DONOR ID NORMALIZATION	28
3.6	BLOOD GROUP NORMALIZATION	29
3.7	REGISTER NORMALIZATION	29
3.8	SCHEMA DIAGRAM FOR BLOOD DONOR FINDER 2.0 FUNCTIONAL DEPENDENCY	30 - 31
4.1	ITERATIVE METHODOLOGY	37
7.1	LOGIN PAGE	55
7.2	HOME PAGE	56
7.3	REGISTRATION PAGE	56
7.4	FIND DONOR PAGE	57
7.5	ADMIN PAGE	57
7.6	ABOUT PAGE	58
10.1	QR CODE	63

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

In an era marked by unprecedented connectivity and mobility, the travel industry stands at the forefront of global interaction, offering individuals the opportunity to explore diverse destinations and cultures. However, navigating the plethora of travel options and packages presented by numerous agencies can often be overwhelming for customers seeking their ideal travel experience. In response to this challenge, the development of a comprehensive BLOOD DONOR FINDER emerges as a pivotal solution, bridging the gap between customers and travel agencies while facilitating seamless view of a variety of options for the customers. In essence, the BLOOD DONOR FINDER not only serves as a solution to the challenges posed by the abundance of travel options but also embodies the transformative potential of technology in revolutionizing the travel industry. By fostering connectivity, simplifying processes, and enhancing customer satisfaction, the System emerges as a cornerstone of modern travel management, driving growth, innovation, and excellence in the ever-expanding global tourism landscape. The travel industry continues to expand, driven by factors such as globalization, increasing disposable income, and a growing desire for experiential travel. As more people seek personalized and convenient travel experiences, platforms like ours will facilitate connections between customers and agencies which is very valuable.

1.2 INTRODUCTION TO JAVA SCRIPT

JavaScript is an important programming language used in the development of our Blood Donor Finder website. It is a client-side scripting language that helps make web pages interactive and dynamic. In our project, JavaScript is used to handle user interactions such as form validation, storing and displaying registered donor details, and managing the donor list section. When a user scans the QR code and opens the website, JavaScript helps control the registration process by checking whether all required fields like name, blood group, and contact number are properly filled. It also enables the dynamic display of registered donors so that users who need blood can easily view the list and contact donors directly. By using JavaScript, our website becomes more responsive, efficient, and user-friendly. Overall, JavaScript plays a key role in improving functionality and ensuring smooth communication between blood donors and recipients in our project.

1.2.1 HISTORY OF JAVASCRIPT

JavaScript is one of the most important programming languages used in web development today. It is mainly used to create interactive and dynamic websites, such as forms, buttons, validation systems, QR-based redirection, and real-time updates just like in your Blood Donor Finder project.

1. Creation of JavaScript (1995)

JavaScript was created in 1995 by Brendan Eich while working at Netscape.

- It was developed in just 10 days.
- Originally, it was named Mocha.
- Later, it was renamed to Live Script.
- Finally, it was named JavaScript to attract attention because Java was popular at that time.

Despite the name, JavaScript is completely different from Java.

2. Early Browser Support

- JavaScript was first introduced in the Netscape Navigator browser.
- Soon after, Microsoft developed its own version called JScript for Internet Explorer. This created compatibility issues between browsers.

3. Standardization – ECMAScript (1997)

- To solve browser conflicts, JavaScript was standardized in 1997 by ECMA International.
- The standardized version was called ECMAScript (ES).

Some important versions:

- ES5 (2009) – Improved stability and browser support.
- ES6 (2015) – Major update with new features like:
 - let and const
 - Arrow functions
 - Classes
 - Modules

4. Modern JavaScript Era

- In 2009, Ryan Dahl created Node.js.
- This allowed JavaScript to run on servers, not just in browsers. After this, JavaScript became a full-stack programming language.

Today, JavaScript is used in:

- Frontend development (React, Angular, Vue)
- Backend development (Node.js)
- Mobile apps
- Games
- Web applications like your Blood Donor Finder system

5. JavaScript Today

JavaScript is now:

- One of the most popular programming languages in the world
- Supported by all modern browsers
- Used by companies like Google, Facebook, and Microsoft
- Continuously updated every year

It plays an important role in:

- Form validation
- Dynamic content updates
- QR code redirection
- Managing donor lists
- Interactive website features

1.2.2 APPLICATION OF JAVASCRIPT

In our **Blood Donor Finder Website**, JavaScript plays a major role in making the website functional, interactive, and user-friendly. Without JavaScript, the website would only display static information. JavaScript helps us manage user input, store donor details, and dynamically display the list of registered donors.

➤ Form Handling

When a user scans the QR code and opens the website, they can register as a blood donor by filling out the registration form. JavaScript collects the entered details such as name, blood group, and phone number when the user clicks the submit button. It processes this data and prepares it for storage.

➤ Form Validation

JavaScript checks whether all required fields are filled correctly. For example, it ensures that the phone number contains only numbers and that no field is left empty. If there is an error, it shows alert messages to guide the user. This improves accuracy and prevents incorrect data submission.

➤ Data Storage (Frontend)

JavaScript stores the registered donor details in the browser using Local Storage. The data is saved even if the page is refreshed. When someone selects the “View Donor List” option, JavaScript retrieves the stored data and displays it on the screen.

➤ Dynamic Donor List Display

When users click on the list option to find blood donors, JavaScript dynamically creates and displays the list of registered donors. It shows their name, blood group, and phone number without reloading the page. This makes the website fast and responsive.

➤ Filtering by Blood Group

JavaScript can also be used to filter donors based on the selected blood group. For example, if someone needs O+ blood, they can choose O+ from a dropdown menu, and JavaScript will display only matching donors.

➤ QR Code Integration

JavaScript can generate or handle the QR code that redirects users to the website. When scanned, it quickly opens the Blood Donor Finder page, making access easy during emergencies.

1.3 INTRODUCTION TO CSS

In our **Blood Donor Finder Website**, CSS (Cascading Style Sheets) is used to design and style the website to make it attractive, user-friendly, and visually organized. While HTML provides the structure of the website and JavaScript adds functionality, CSS is responsible for the appearance and layout of the web pages.

CSS helps us design the registration form, donor list page, buttons, headings, and navigation menu in a clean and professional way. It allows us to change colours, fonts, background styles, spacing, and alignment. For example, we can use CSS to design a red-themed layout that represents blood donation, style the submit button with hover effects, and create properly spaced input fields for better readability.

In this project, CSS is also used to make the website responsive, meaning it works properly on mobile phones, tablets, and computers. Since users scan the QR code using their mobile devices, responsive design is very important. CSS media queries help adjust the layout based on screen size so that the website looks good on all devices.

Additionally, CSS improves user experience by adding visual effects such as button animations, smooth transitions, shadows, and borders. These small design elements make the website more interactive and attractive.

In conclusion, CSS plays a vital role in our Blood Donor Finder project by enhancing the visual presentation, improving user experience, and ensuring the website is responsive and easy to use.

1.3.1 HISTORY OF CSS

CSS (Cascading Style Sheets) was developed to improve the design and presentation of web pages. In the early days of the internet, web pages were created using only HTML. HTML was mainly used for structuring content, but it had very limited styling options. As websites became more popular, developers needed a better way to control layout, colours, fonts, and overall design.

CSS was first proposed in 1994 by Håkon Wium Lie. Later, it was developed by the World Wide Web Consortium (W3C) to standardize web design. The first official version, CSS1, was released in 1996. It introduced basic styling features such as font properties, colours and spacing.

In 1998, CSS2 was introduced with advanced features like positioning, z-index, and media types. This allowed developers to create more complex and visually appealing web pages. Over time, CSS continued to improve. CSS3, released in the early 2000s, introduced many powerful features such as animations, transitions, gradients, shadows, flexbox, and media queries. These features made modern responsive web design possible.

Today, CSS is an essential part of web development. It works together with HTML and JavaScript to create attractive, responsive, and interactive websites. In our Blood Donor Finder project, CSS helps design the registration form, donor list page, buttons, and layout, making the website visually appealing and user-friendly.

1.3.2 APPLICATIONS OF CSS

In our **Blood Donor Finder Website**, CSS (Cascading Style Sheets) is used to design and enhance the appearance of the website. While HTML provides the structure and JavaScript provides functionality, CSS is responsible for making the website visually attractive, organized, and easy to use.

➤ **Designing the Registration Form**

CSS is used to style the blood donor registration form by adjusting the layout, input field sizes, fonts, colours, and spacing. It ensures that the form looks clean and professional. Buttons such as “Register” are styled with background colours, rounded corners, and hover effects to improve user interaction.

➤ **Styling the Donor List**

When users select the “View Donor List” option, CSS helps present the list in a clear and readable format. It can display donor details (name, blood group, and phone number) in a table or card layout with proper alignment, borders, and spacing.

➤ **Colour Theme and Visual Design**

CSS allows us to apply a suitable colour theme, such as red and white, which represents blood donation. It improves the overall appearance of the website and makes it visually meaningful and attractive.

➤ **Responsive Design**

Since users access the website by scanning a QR code on their mobile phones, CSS is used to make the website responsive. With the help of media queries, the layout automatically adjusts to different screen sizes like mobiles, tablets, and desktops.

➤ **Improving User Experience**

CSS adds interactive features such as hover effects, transitions, shadows, and animations. These features make buttons and menus respond visually when users interact with them, creating a smooth and user-friendly experience.

➤ **Layout and Page Structure**

CSS controls the alignment of elements using properties like flexbox or grid. It helps in organizing headings, navigation menus, forms, and lists properly so that the website looks structured and professional.

1.4 INTRODUCTION TO HTML

HTML (Hyper Text Markup Language) is the basic building block of our **Blood Donor Finder Website**. It is used to create the structure and content of the web pages. In simple words, HTML forms the skeleton of the website, while CSS designs it and JavaScript adds functionality.

In our project, HTML is used to create the main layout of the website that opens when a user scans the QR code. It defines all the elements such as headings, paragraphs, buttons, forms, input fields, and links. For example, the blood donor registration form is created using HTML form elements like `<form>`, `<input>`, `<label>`, and `<button>`. These elements allow users to enter their name, blood group, and phone number.

HTML is also used to create the “View Donor List” page, where registered donors’ details such as name, blood group, and contact number are displayed. It organizes the data in a structured way using tables or div sections so that the information is clear and easy to read.

Additionally, HTML helps in creating navigation options like buttons or menu links that allow users to switch between the registration page and the donor list page. It ensures that the website content is properly arranged and accessible on different devices.

1.4.1 HISTORY OF HTML

HTML, or **Hyper Text Markup Language**, is the foundational language used to create web pages and web applications, including our **Blood Donor Finder Website**. It was first developed in **1989** by **Tim Berners-Lee**, a British scientist, while working at CERN. His goal was to create a system that allowed documents to be linked across different computers using hyperlinks, which became the basis of the World Wide Web.

The first version of HTML was very simple and allowed only basic elements such as headings, paragraphs, lists, and links. In **1995**, HTML 2.0 was standardized, which introduced more structured elements and forms, making it possible to collect user input. This was an important step that later allowed projects like ours to create registration forms for blood donors.

Over time, HTML evolved with new versions: **HTML 3.2** (1997) and **HTML 4.01** (1999) introduced advanced features such as tables, styles, and multimedia elements. The most significant modern version is **HTML5**, released in 2014, which added features for interactive web applications, multimedia support (audio, video), improved form handling, and better integration with CSS and JavaScript.

1.4.2 APPLICATION OF HTML

In our Blood Donor Finder Website, HTML (Hypertext Markup Language) is used to create the structure and layout of the web pages. It provides the foundation on which CSS and JavaScript work to make the website functional, interactive, and visually appealing.

➤ Creating the Registration Form

HTML is used to design the blood donor registration form, which allows users to enter their name, blood group, phone number, and other details. Elements like `<form>`, `<input>`, `<label>`, and `<button>` are used to create input fields and submission buttons. This allows users who want to donate blood to register quickly and easily.

➤ Displaying Donor List

The “View Donor List” page is built with HTML to display registered donors in a structured way. Tables (`<table>`, `<tr>`, `<td>`) or div sections are used to organize donor information, such as name, blood group, and contact number, so users can easily find and call the right donor.

➤ Navigation and Links

HTML provides navigation elements like buttons and hyperlinks (<a>) that allow users to switch between the registration page and the donor list page. This ensures smooth user interaction and makes the website easy to use.

➤ Organizing Content

HTML ensures that all content, including headings, paragraphs, and sections, is properly structured. It makes the website readable and accessible on different devices, whether a mobile phone, tablet, or computer.

➤ Integration with CSS and JavaScript

HTML acts as the skeleton of the website, which is styled using CSS and made interactive with JavaScript. For example, the registration form created with HTML is validated with JavaScript, and the donor list is dynamically displayed and filtered using JavaScript, while CSS adds colours, fonts.

1.5 Theory and Concepts

➤ Introduction

The **Blood Donor Finder Website** is a web-based application designed to help people either **donate blood** or **find blood donors** in case of emergencies. Users can access the website by scanning a **QR code**, which opens the site directly on their device. On the website, users have two main options:

1. **Register as a donor** – If they want to donate blood.
2. **View donor list** – If they need blood, they can see the list of registered donors along with their blood group and contact number to make a call.

This project combines **HTML, CSS, and JavaScript** to create a fully functional, interactive, and user-friendly application.

Concepts Used in the Project:

➤ **HTML (Hypertext Markup Language)**

- HTML is used to create the **structure** of the website.
- It defines elements like headings, forms, buttons, input fields, and tables.
- Example in the project: The **registration form** and the **donor list page** are built using HTML tags such as <form>, <input>, <button>, <table>, <tr>, <td>, etc.
- HTML acts as the **skeleton** of the website and makes the content organized and readable.

➤ CSS (Cascading Style Sheets)

- CSS is used to **style and design** the website, making it visually appealing and user-friendly.
- Example in the project:
 - The registration form, buttons, and donor list are styled for better readability.
 - Colores like red and white are used to represent blood donation themes.
 - Responsive design ensures the website works well on **mobile phones, tablets, and desktops**.
- CSS improves user experience through hover effects, smooth transitions, and proper alignment of elements.

➤ JavaScript

- JavaScript adds **interactivity and functionality** to the website.
- Example in the project:
 - Validates user input in the registration form.
 - Stores donor information in **Local Storage** (frontend storage).
 - Dynamically displays the donor list without refreshing the page.
 - Filters donors by blood group for easy search.
 - Can generate or handle the QR code dynamically if needed.
- JavaScript ensures the website is **interactive, dynamic, and functional**.

➤ QR Code Integration

- QR code is used for **quick access** to the website.
- When scanned, it redirects users directly to the Blood Donor Finder website, eliminating the need to type a URL manually.
- Useful in **emergency situations** where quick access is critical.

➤ Data Storage (Frontend Level)

- Donor information is stored on the user's browser using **Local Storage** in JavaScript.
- The data includes **name, blood group, and contact number**.
- This allows the donor list to remain available even after page refresh.

- For advanced applications, a **backend database** like Firebase or MySQL could be used for centralized storage.

➤ **Interactive User Interface**

- The UI is designed to be **simple, clear, and responsive**.
- Buttons, forms, and navigation menus respond immediately to user actions.
- JavaScript and CSS together make the website interactive and easy to use.

➤ **Real-Time Updates (Advanced Concept)**

- In advanced implementations, **real-time updates** can show new donor registrations immediately.
- Technologies like Firebase or APIs can push updates instantly without page refresh.
- Ensures that users always see the latest donor information, which is crucial during emergencies.

➤ **Flow of the Project**

1. User scans the QR code → Website opens.
2. User chooses an option:
 - **Register as a donor** → Fill form → Submit → Data stored.
 - **Find a donor** → View donor list → Call the donor.
3. Website dynamically displays donor information using JavaScript.
4. CSS ensures the website is responsive and visually appealing.

➤ **Advantages of the Project**

- Quick access via QR code.
- Easy registration for donors.
- Instant availability of donor information for people in need.
- Interactive, responsive, and mobile-friendly design.

1.6 INTRODUCTION TO DATABASE MANAGEMENT SYSTEM

Databases and database technology have a major impact on the growing use of computers. It is fair to say that databases play a critical role in almost all areas where computers are used, including business, electronic commerce, engineering, medicine, genetics, law, education, and library science. The word database is so commonly used that User must begin by defining what a database is. Our initial definition is quite general. A database is a collection of related data.¹ By data, User mean known facts that can be recorded and that have implicit meaning. For example, consider the names, telephone numbers, and addresses of the people you know. You may have recorded this data in an indexed address book or you may have stored it on a hard drive, using a personal computer and software such as Microsoft Access or Excel. This collection of related data with an implicit meaning is a database. The preceding definition of database is quite general; for example, User may consider the collection of words that make up this page of text to be related data and hence to constitute a database. However, the common use of the term database is usually more restricted.

A database has the following implicit properties:

- A database represents some aspect of the real world, sometimes called the mini world or the universe of discourse. Changes to the inworld are reflected in the database.
- A database is a logically coherent collection of data with some inherent meaning. A random assortment of data cannot correctly be referred to as a database
- A database is designed, built, and populated with data for a specific purpose. It has an intended. group of users and some preconceived applications in which these users are interested.

A database management system (DBMS) is a collection of programs that enables users to create and maintain a database. The DBMS is a general-purpose software system that facilitates the processes of defining, constructing, manipulating, and sharing databases among various users and applications. Defining a database involves specifying the data types, structures, and constraints of the data to be stored in the database. The database definition or descriptive information is also stored by the DBMS in the form of a database catalogue or dictionary; it is called meta-data. Constructing the database is the process of storing the data on some storage medium that is controlled by the DBMS. Manipulating a database includes functions such as querying the database to retrieve specific data, updating the database to reflect changes in the inworld, and generating reports from the data. Sharing a database allows multi CWC users and programs to access the database simultaneously.

1.7 History of DBMS

In 1959, the TX-2 computer was developed at MIT's Lincoln Laboratory. The TX-2 integrated a number of new man-machine interfaces. A light pen could be used to draw sketches on the computer using Ivan Sutherland's revolutionary Sketchpad software. Using a light pen, Sketchpad allowed one to draw simple shapes on the computer screen, save them and even recall them later. The light pen itself had a small photoelectric cell in its tip. This cell emitted an electronic pulse whenever it was placed in front of a computer screen and the screen's electron gun fired directly at it. By simply timing the electronic pulse with the current location of the electron gun, it was easy to pinpoint exactly where the pen was on the screen at any given moment. Once that was determined, the computer could then draw a cursor at that location. Also, in 1961 another student at MIT, Steve Russell, created the first video game, E. Zaxxon, a scientist at Bell Telephone Laboratory (BTL), created a film called "Simulation of a two-gravity attitude control system" in 1963. During 1970s, the first major advance in 3D computer graphics was created at UU by these early pioneers, the hidden-surface algorithm. In order to draw a representation of a 3D object on the screen, the computer must determine which surfaces are "behind" the object from the viewer's perspective, and thus should be "hidden" when the computer creates (or renders) the image. In the 1980s, artists and graphic designers began to see the personal computer, particularly the Commodore Amiga and Macintosh, as a serious design tool, one that could save time and draw more accurately than other methods. In the late 1980s, SGI computers were used to create some of the first fully computer-generated short films at Pixar. The Macintosh remains a highly popular tool for computer graphics among graphic design studios and businesses. Modern computers, dating from the 1980s often use graphical user interfaces (GUI) to present data and information with symbols, icons and pictures, rather than text. Graphics are one of the five key elements of multimedia technology. 3D graphics became more popular in the 1990s in gaming, multimedia and animation. In 1996, Quake, one of the first fully 3D games, was released. In 1995, Toy Story, the first full-length computer-generated animation film, was released in cinemas worldwide. Since then, computer graphics have only become more detailed and realistic, due to more powerful graphics hardware and 3D modeling software.

1.7.1 APPLICATIONS OF DBMS

Applications where we use Database Management Systems are:

- **Telecom:** There is a database to keep track of the information regarding calls made, network usage, customer details etc. Without the database systems it is hard to maintain that huge amount of data that keeps updating every millisecond.
- **Industry:** Where it is a manufacturing unit, warehouse or distribution Centre, each one needs a database to keep the records of ins and outs. For example, distribution Centre should keep a track of the product units that supplied into the Centre as well as the products that got delivered out from the distribution Centre on each day; this is where DBMS comes into picture.
- **Banking System:** For storing customer info, tracking day to day credit and debit transactions, generating bank statements etc. All this work has been done with the help of Database management systems.
- **Education sector:** Database systems are frequently used in schools and colleges to store and retrieve the data regarding student details, staff details, course details, exam details, payroll data, attendance details, fees detail etc. There is a hell lot amount of inter-related data that needs to be stored and retrieved in an efficient manner.
- **Online shopping:** You must be aware of the online shopping websites such as Amazon, Flipkart etc. These sites store the product information, your addresses and preferences, credit details and provide you the relevant list of products based on your query. All this involves a Database management system.

1.7.2 Theory and Concepts

- **Inheritance:** In object-oriented programming, inheritance is when an object or class is based on another object or class, using the same implementation (inheriting from an object or class) or specifying a new implementation to maintain the same behavior (realizing an interface). Such an inherited class is called a subclass of its parent class or super class.
- **Encapsulation:** In object-oriented programming, encapsulation is a mechanism of binding the data, and the functions together in a class and use them by creating an object of that class.
- **Data Abstraction:** Data abstraction refers to, providing only essential information to the outside world and hiding their background details, i.e., to represent the needed information in program without presenting the implementation details. Data abstraction is a programming (and design) technique that relies on the separation of interface and implementation.

1.8 REQUIREMENT SPECIFICATION

Hardware Requirements

1. Processor

- **Minimum:** Intel Core i3-1005G1 CPU @ 1.20GHz or equivalent.
- **Recommended:** Intel Core i5 or higher for better performance.

2. RAM

- **Minimum:** 8 GB.
- **Recommended:** 16 GB or more, especially if running additional services or for a higher number of concurrent users.

3. Hard Disk

- **Minimum:** 500 GB HDD.
- **Recommended:** 1 TB or more, SSD for faster data access and improved performance.

4. Input Devices

- **Minimum:** Standard keyboard and mouse.

5. Output Devices

- **Minimum:** Monitor with at least 1080p resolution.
- **Recommended:** Monitor with higher resolution for better user experience.

Software Requirements

1. Operating System

- **Minimum:** Windows 10, macOS 10.13 High Sierra, or a popular Linux distribution such as Ubuntu 18.04.
- **Recommended:** Latest stable version of the operating system for better security and support.

2. Web Server

- **Minimum:** Flask development server (for small-scale or testing purposes).
- **Recommended:** A production-ready web server like Nginx or Apache for better performance and security.

3. Database

- **Minimum:** MySQL 8.0.36.
- **Recommended:** Latest stable version of MySQL or an equivalent database management system.

4. Programming Language

- **Python Version:** Python 3.11.

5. Integrated Development Environment (IDE)

- **Minimum:** Visual Studio Code.
- **Recommended:** Any other IDE or text editor the developer is comfortable with, such as PyCharm or Sublime Text.

CHAPTER 2

SYSTEM STUDY

2.1 FEATURES OF THE SYSTEM

Our Blood Donor Finder Website is designed to make blood donation and blood search fast, easy, and efficient. Below are the key features of the system:

➤ **QR Code Access:**

- The website can be accessed instantly by scanning a QR code.
- Eliminates the need to type the website URL manually.
- Makes the system fast and convenient, especially during emergencies.

➤ **Donor Registration:**

Users who want to donate blood can register online.

- The registration form collects:
 - Name
 - Blood group
 - Contact number
 - (Optional) Location
- JavaScript validation ensures that all required fields are filled correctly.

➤ **Donor List Display:**

Users who need blood can view a list of registered donors.

- The list includes:
 - Donor name
 - Blood group
 - Contact number
- Allows users to directly contact donors via phone.

➤ **Interactive and Dynamic Interface:**

- The website responds instantly to user actions.
- The donor list updates dynamically when a new donor is registered.
- Buttons, forms, and menus have visual feedback to improve user experience.

➤ **Blood Group Filtering (Optional):**

- Users can filter the donor list based on the required blood group.
- Makes it faster to find donors who match specific needs.

- **Frontend Data Storage:**
 - Donor details are stored in Local Storage on the browser.
 - Data remains available even after refreshing the page.
 - For advanced versions, data can be stored in a database for centralized access.
- **Responsive Design:**
 - The website works on mobile phones, tablets, and desktops.
 - CSS ensures that the layout adjusts automatically to different screen sizes.
 - Essential since most users will access the site via QR code on their smartphones.
- **Emergency Friendly:**
 - Designed for quick access and easy navigation.
 - Reduces the time needed to find blood donors in urgent situations.
- **Expandable System (Future Features):**
 - Real-time updates for new donors.
 - Location-based donor search.
 - Admin panel to manage donors and requests.

2.2 SOFTWARE DESCRIPTION

FRONT END :

➤ **HTML (Hypertext Markup Language)**

Purpose: HTML is the fundamental building block of web pages. It defines the structure and layout of a webpage by using a variety of tags and elements.

Key Features:

- **Tags and Elements:** HTML uses tags like <div>, <p>, <a>, <table>, and many others to structure the content. Each tag serves a specific purpose.
- **Forms:** HTML forms are used to collect user inputs, such as text fields, radio buttons, checkboxes, and submit buttons.
- **Hyperlinks:** Using the <a> tag, HTML enables navigation between different pages of the web application.
- **Multimedia:** HTML supports embedding images, videos, and audio with tags like , <video>, and <audio>.

Role in the System:

- **Page Layout:** Defines the structure of various pages such as the home page, login/signup pages, package details, and user dashboards.
-

- **Forms:** Used for agency and customer registration, login forms, package addition forms, etc.
- **Tables:** Displaying lists of agencies, customers, packages, and bookings.

CSS (Cascading Style Sheets) :

Purpose: CSS is used to style HTML elements. It defines how HTML elements are displayed, allowing for the separation of content from presentation.

Key Features:

- **Selectors:** CSS selectors are used to select the HTML elements you want to style.
- **Box Model:** CSS defines the spacing, border, and padding of HTML elements.
- **Flexbox and Grid:** Modern layout systems for creating responsive and flexible designs.
- **Media Queries:** Used to apply different styles for different devices and screen sizes, making web pages responsive.

Role in the System:

- **Visual Design:** Ensures a consistent look and feel across the web application. It styles headers, footers, navigation bars, forms, buttons, and tables.
- **Responsive Design:** Ensures that the web application is usable on different devices, including desktops, tablets, and smartphones.
- **User Experience:** Improves the user experience by providing visual feedback on user interactions, such as hover effects and form validations.

JAVASCRIPT :

Purpose: JavaScript is a programming language that enables interactive web pages. It is executed on the client-side (in the user's browser).

Key Features:

- **DOM Manipulation:** JavaScript can read and alter the Document Object Model (DOM), allowing dynamic content updates.
- **Event Handling:** JavaScript can respond to user actions like clicks, form submissions, and mouse movements.
- **AJAX:** Asynchronous JavaScript and XML (AJAX) allows for asynchronous data loading without reloading the entire page.
- **Libraries and Frameworks:** Tools like jQuery, React, or Angular provide additional functionalities and ease of use.

Role in the System:

- **Form Validation:** Ensures user inputs are valid before submission to the server, reducing server load and improving user experience.
- **Dynamic Content:** Updates parts of the web page dynamically, such as updating package details or user profiles without a full page reload.
- **Interactivity:** Enhances user interaction with elements like dropdowns, modals, and real-time notifications.

PACKAGE.JSON :

Package. Json is a main configuration file in a Node.js application that contains information about the project and manages all the required dependencies, scripts, and settings needed to run the backend system.

Key Features**1. Project Information**

- Stores project name, version, and description
- Helps identify the application

2. Dependency Management

- Lists all required libraries (like Express, Mongoose)
- Allows installation using npm install

3. Script Management

- Defines commands to run the project
- Example:
 - npm start → starts server
 - npm run dev → runs with auto-reload

4. Version Control

- Maintains specific versions of packages
- Ensures consistency across different systems

Role in the System (Blood Donor Finder)**1. Backend Setup Controller**

- Initializes and manages the backend server

2. Enables Donor Registration

- Supports APIs to store donor details:
 - Name
 - Blood group
 - Phone number s

3. Handles Donor Listing

- Helps fetch and display registered donors

4. Connects Frontend with Backend

- Uses middleware (like CORS) to allow communication with website (QR code page)

SERVER.JS :

- server.js is the main file of your Node.js backend.
- It starts the server, handles API routes, and connects your backend to the database.
- For your Blood Donor Finder project, it manages:
 - Donor registration
 - Donor list retrieval
 - Communication with the frontend (website)

Key Features of server.js

- Server Initialization → Creates and runs the backend server using Express.
- Route Handling → Handles API endpoints like /register and /donors.
- Database Connection → Connects to MongoDB to store donor data.
- Middleware Support → Uses tools like cors (for frontend-backend communication) and body-parser (to read form data).
- Environment Configuration → Reads .env file for database URL, port, and other secrets.

Role in the System (Blood Donor Finder)

- Starts the backend server for the website.
- Handles donor registration and saves data in the database.
- Provides API to fetch the list of donors.
- Connects frontend (QR code website) with backend.
- Manages database connection and ensures smooth communication.

2.3 FUNCTIONAL REQUIREMENT

Functional requirements describe the specific behavior or functions of the system. For the System, these include:

➤ **QR Code Access**

- The system shall generate a **QR code** linked to the website.
- When users scan the QR code, they shall be redirected to the homepage of the blood donor website.

➤ **Homepage**

- The system shall display:
 - Project introduction (purpose of blood donation).
 - Two main options:
 - **Register as Donor**
 - **Find Blood Donors (List View)**

➤ **Donor Registration**

- The system shall allow users to register as blood donors by filling a form.
- The form shall include:
 - Name
 - Blood Group (A+, A-, B+, etc.)
 - Phone Number
 - Location (optional but useful)
- The system shall validate:
 - Required fields must not be empty.
 - Phone number format must be valid.
- The system shall store the registered donor details in a database.

➤ **Donor List Viewing**

- The system shall display a list of all registered donors.
- The list shall include:
 - Name
 - Blood Group

- Contact Number
- The system shall allow users to:
 - View all donors.
 - Search/filter donors by blood group.
- **Search Functionality**
 - The system shall provide a search option.
 - Users shall be able to:
 - Search donors by blood group.
 - (Optional) Filter by location.
- **Contact Donor**
 - The system shall allow users to view donor phone numbers.
 - Users can directly call donors using the displayed contact number.

2.4 NON-FUNCTIONAL REQUIREMENT

Non-functional requirements describe the quality attributes, performance criteria, and constraints of the system. For the System, these include:

- **Performance:**
 - The system shall load pages quickly and respond fast to user actions.
 - **Explanation:** When users scan the QR code or search for donors, the website should open and display results within a few seconds, especially during emergencies.
- **Usability:**
 - The system shall be easy to use and user-friendly.
 - **Explanation:** Users should easily understand how to register or find donors without confusion. The interface should be simple, clear, and accessible to all users.
- **Compatibility (Responsiveness):**
 - The system shall work on different devices and screen sizes.
 - **Explanation:** Since users access the site through QR codes, the website must work properly on mobile phones, tablets, and desktops.
- **Security:**
 - The system shall protect user data from unauthorized access.
 - **Explanation:** Donor details like phone numbers must be stored securely and should not be misused or accessed by unauthorized users.

➤ **Reliability:**

- The system shall be available and function properly at all times.
- **Explanation:** The website should not crash and should be available whenever users need it, especially in emergency situations.

➤ **Error Handling:**

- The system shall handle errors properly and show meaningful messages.
- **Explanation:** If users enter incorrect data or something goes wrong, the system should display clear error messages instead of crashing.

➤ **Scalability (Optional but Good):**

- The system shall handle increasing number of users and data.
- **Explanation:** As more donors register, the system should still work smoothly without slowing down.

CHAPTER 3

SYSTEM DESIGN

3.1 ER DIAGRAM

An **Entity–Relationship Diagram (ER Diagram)** is a visual representation used in database design to show how data entities are related to each other. In the **Blood Donor Finder system**, the ER diagram helps in organizing and structuring the data required to manage donors, blood groups, and registration details efficiently.

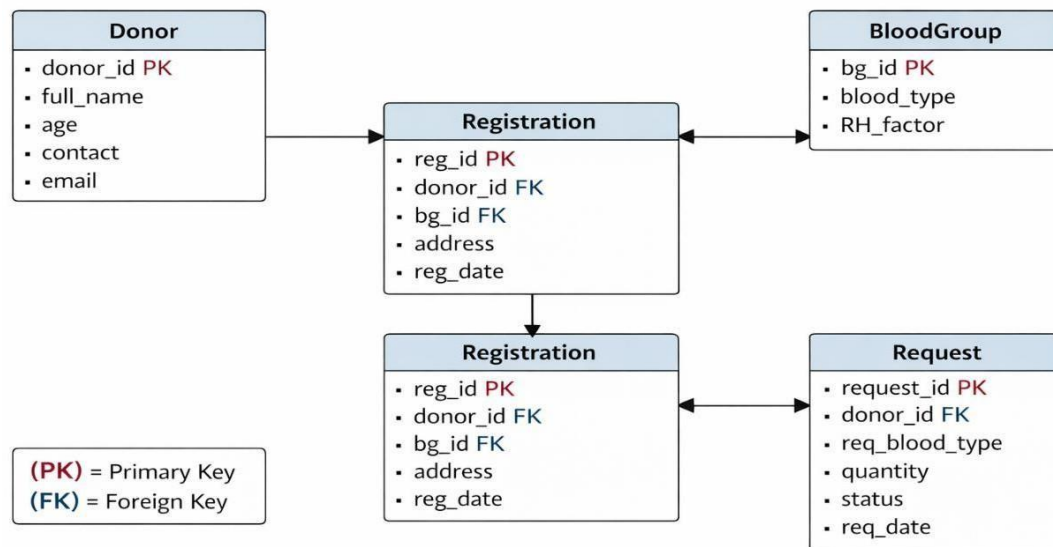


Figure 3.1: ER Diagram for Blood Donor Finder

The Agencies Have to insert tour packages, which will be viewed by the customers. The customers then can opt any of the package. After opting the package, the Agency will receive the customers information. When customer opts a package, Agency will then Assign a Guide for the package.

3.2 MAPPING FROM ER DIAGRAM TO SCHEMA DIAGRAM

STEP-1: Mapping of Regular Entities:

To map the Regular Entities of your Blood Donor Finder project from an ER diagram to a relational schema, you first identify the core independent objects that drive the system. In this context, regular entities—such as Donor, Recipient, and Blood Bank are those that exist independently and possess a unique identifier, or Primary Key.

BLOOD DONATION DATABASE SCHEMA

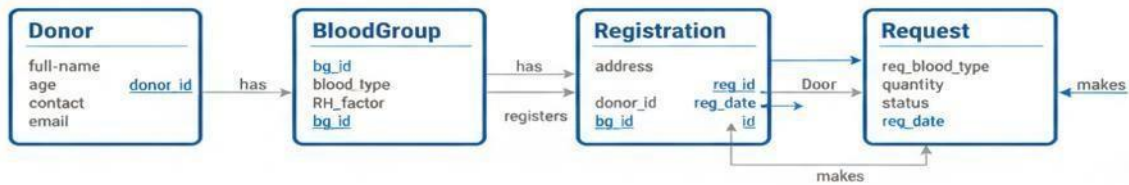


Figure 3.2: Mapping of Regular Entities

STEP-2: Mapping of Weak Entities:

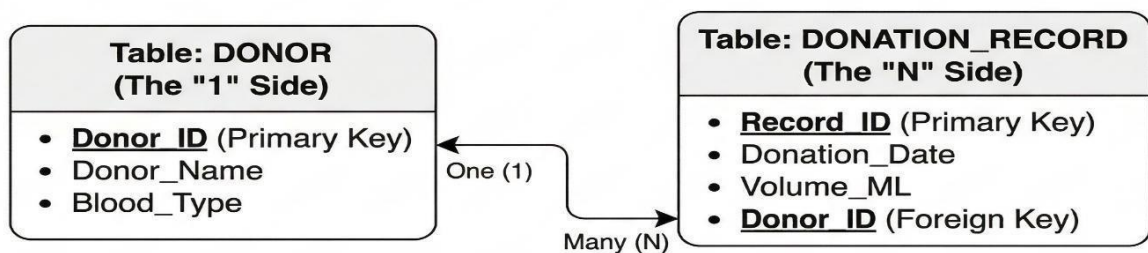
In your Blood Donor Finder system, Weak Entities are those that cannot be uniquely identified by their own attributes alone and depend on a "strong" or "owner" entity for their existence. A typical example would be Donation History or Medical Records, which only exist if a Donor exists.

STEP-3: Mapping of Binary 1:1 Relationships:

In the Blood Donor Finder system, mapping a Binary 1:1 Relationship involves linking two entities where each instance of the first entity is associated with exactly one instance of the second. A common example in your project might be the relationship between a Donor and their specific donor card.

STEP-4 :Mapping of Binary 1:N Relationships:

In your Blood Donor Finder project, a Binary 1:N (One-to-Many) Relationship occurs when one instance of an entity is associated with multiple instances of another. A clear example is the relationship between a Blood Bank and its Donors: a single blood bank can manage many donors, but each donor is typically registered at one primary blood bank for a specific donation event.



Mapping: PK (Donor) to FK (Donation_Record)

Figure 3.3: Mapping of Binary 1:N Relationships

STEP-5: Mapping of Binary M:N Relationships:

In your Blood Donor Finder project, a Binary M:N (Many-to-Many) Relationship occurs when multiple instances of one entity can be associated with multiple instances of another. A classic example is the relationship between Donors and Blood Requests.

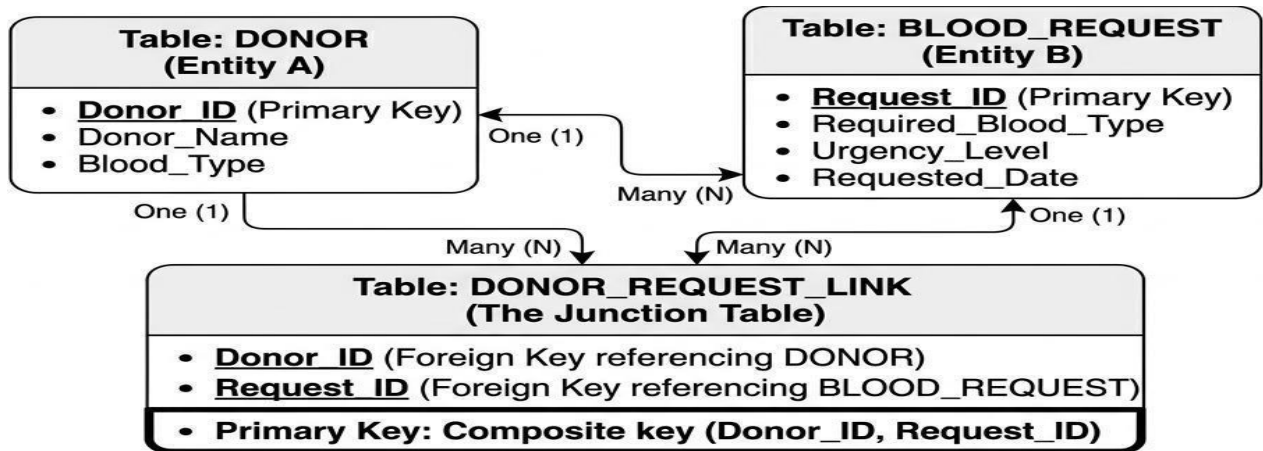


Figure 3.4: Mapping of Binary M:N Relationships

STEP-6: Mapping of Multivalued attributes:

Multivalued attributes in an ER diagram represent attributes that can hold multiple values for a single entity instance. They are depicted with a double ellipse and typically require a separate entity to resolve them into separate instances. The above ER diagram has no multivalued attributes.

STEP-7: Mapping of N-Ary relationships:

In your Blood Donor Finder project, an N-Ary relationship (specifically a Ternary or 3-way relationship) occurs when three different entities are linked simultaneously by a single action. A common example is the relationship between a Donor, a Recipient, and a Hospital during a specific Transaction

3.3 NORMALIZATION

DONOR_ID:

Did $\rightarrow$ {Dtype, name, Dage, Dcontact, Demail}

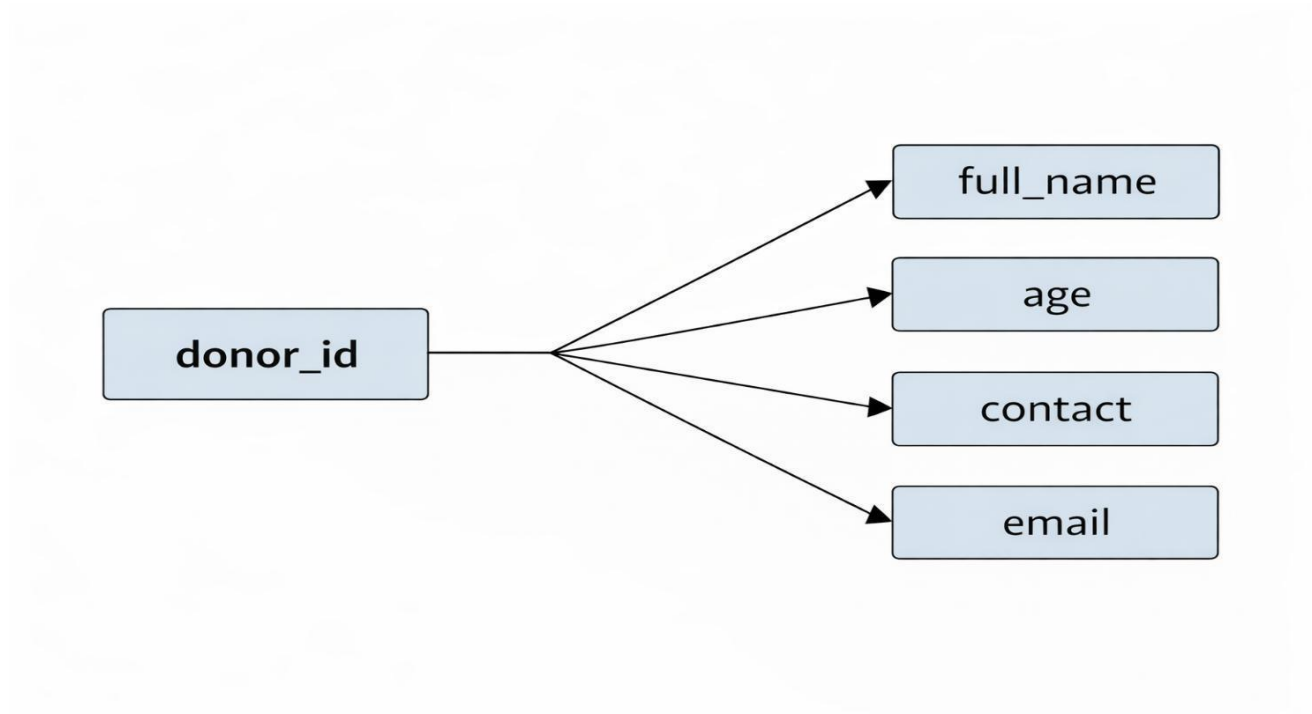


Figure 3.5 Donor_id normalization

1NF: The Functional Dependency is in 1NF since all the attributes are atomic. **2NF:** The Functional Dependency is in 2NF since all the attributes solely depend on the whole primary key, eliminating any partial dependencies.

3NF: The Functional Dependency is in 3NF since there does not exist any transitive dependencies.

Blood Group ID:

BG_id $\rightarrow$ {Btype, RH_factor}

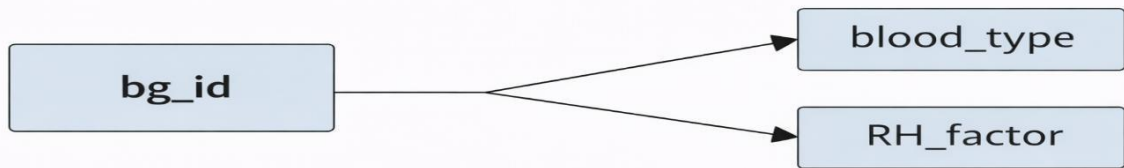


Figure 3.6 Blood Group normalization

1NF: The Functional Dependency is in 1NF since all the attributes are atomic. **2NF:** The Functional Dependency is in 2NF since all the attributes solely depend on the whole primary key, eliminating any partial dependencies.

3NF: The Functional Dependency is in 3NF since there does not exist any transitive Dependencies.

Register ID

$Rid \rightarrow \{Donar_id, BG.id, Daddress\}$

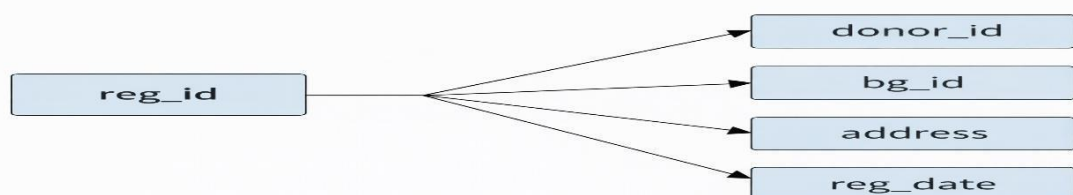


Figure 3.7 Register ID normalization

1NF: The Functional Dependency is in 1NF since all the attributes are atomic.

2NF: The Functional Dependency is in 2NF since all the attributes solely depend on the whole primary key, eliminating any partial dependencies.

3.4 ASSUMPTIONS

- All registered blood banks are assumed to be verified and legally authorized entities.
- The system assumes that donors will provide honest and up-to-date medical history.
- Availability of blood units is assumed to be updated in real-time by the respective blood banks.
- The system assumes a stable internet connection for users to access the "Finder" location services.
- A donor is assumed to be ineligible for a new donation for a specific period after a successful transaction.
- Recipients are assumed to have a valid medical prescription or hospital requirement for blood requests.
- The database assumes that every Donor_ID and Bank_ID is unique and non-null.

3.5 SCHEMA DIAGRAM

A **Schema Diagram** for your Blood Donor Finder project serves as the logical blueprint that translates your conceptual ER model into a structured database environment. Unlike a basic flowchart, this diagram explicitly defines the organization of data into tables, specifying the exact attributes (columns), data types, and the strict constraints that maintain data integrity. At its core, the schema organizes independent entities like **Donors**, **Recipients**, and **Blood Banks** into primary tables, while using **Foreign Keys** to map the essential relationships between them—such as which donor belongs to which bank or which medical records are tied to a specific user.

BLOOD DONATION DATABASE SCHEMA

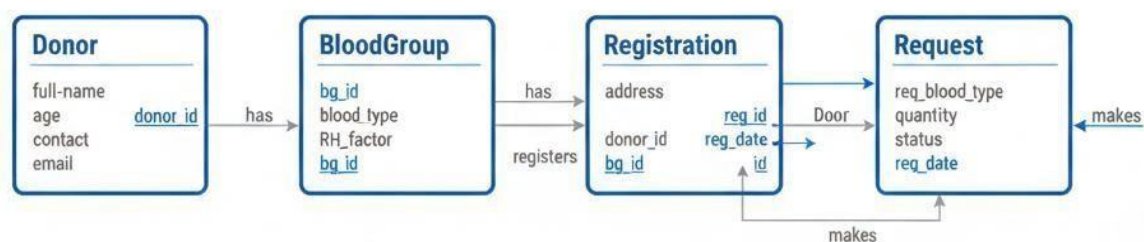


Figure 3.8: Schema Diagram for Blood Donor Finder

3.5.1 FUNCTIONAL DEPENDENCY

In the context of your Blood donor finder project, Functional Dependency (FD) is a constraint that describes the relationship between attributes in a database table. A functional dependency, denoted as $X \rightarrow Y$, means that the value of attribute X uniquely determines the value of attribute Y . For your website to operate without data errors, establishing these dependencies is crucial for Normalization. Without clearly defined FDs, your system might suffer from update anomalies, such as changing a donor's phone number in one record but forgetting to update it in another where the same donor appears.

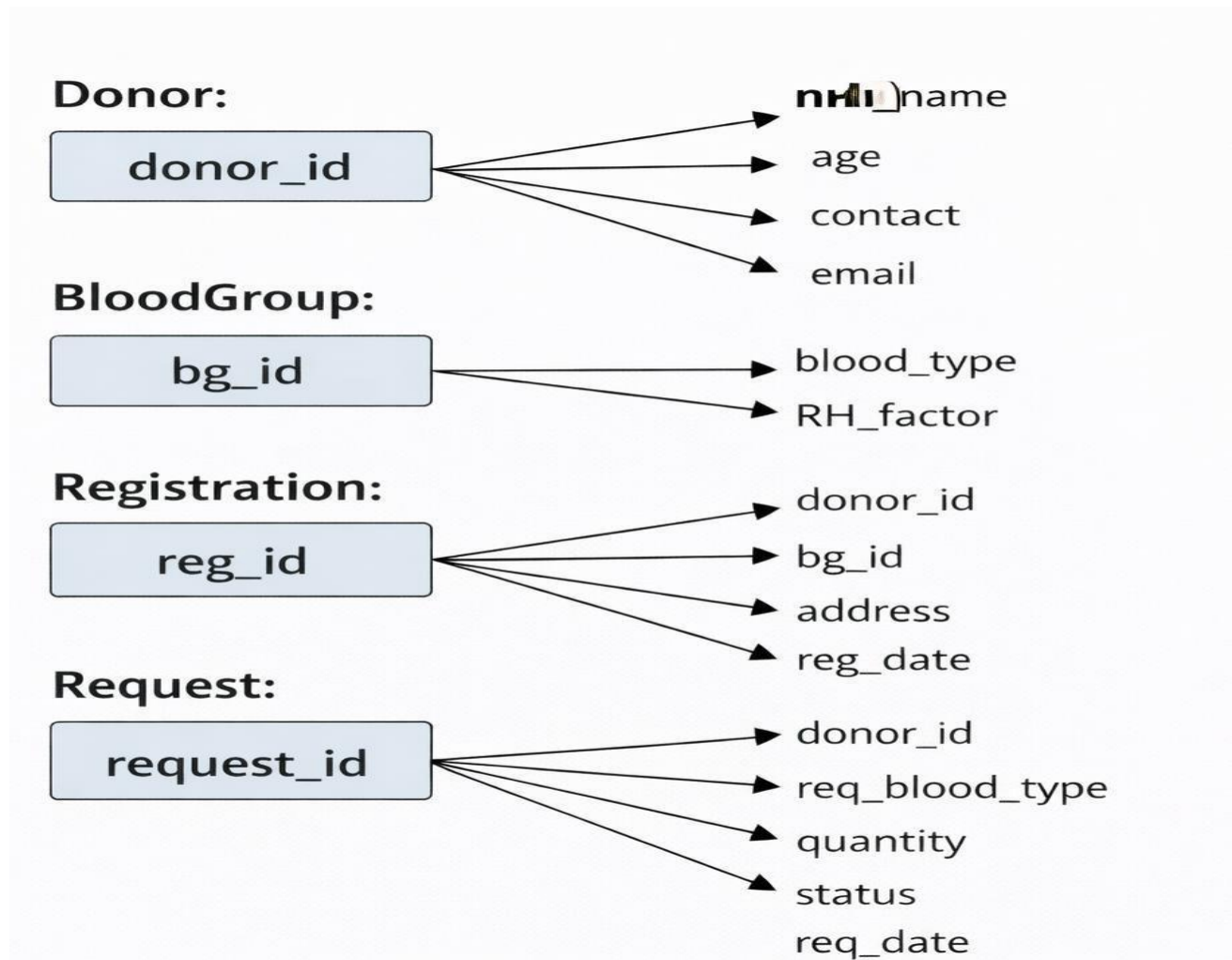


Figure 3.8: Functional Dependency Diagram for Blood Donor Finder

3.6 INPUT DESIGN OF THE SYSTEM

Input design is a critical part of system design that focuses on how data is entered into the system. Effective input design improves user interaction and data accuracy. For the Blood Donor Finder system, various input forms are designed to collect information from users, including donor. Below is an outline of the key input forms and their design considerations.

DONOR LOGIN FORM:

Purpose: To allow donors to register on the platform.

Fields:

- **Donor(d_id):** A unique identifier for the donor.
 - ❖ **Input Type:** Text
 - ❖ **Validation:** Required, alphanumeric, unique
- **Donor Name (d_name):** The name of the Blood Donor
 - ❖ **Input Type:** Text
 - ❖ **Validation:** Required, alphabetic
- **Donor Address (d_address):** The address of the Blood Donor.
 - ❖ **Input Type:** Text
 - ❖ **Validation:** Required
- **Donor Contact (d_contact):** Contact number of the blood donor.
 - ❖ **Input Type:** Text
 - ❖ **Validation:** Required, numeric, valid phone number format

Design Considerations:

- ❖ **Clear Labels:** Ensure each field is clearly labeled.
- ❖ **Placeholder Text:** Provide examples in the placeholder text to guide the user.
- ❖ **Validation Messages:** Display error messages for invalid inputs.

Example HTML:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Blood Donor Finder</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="site-header">
    <div class="header-shell">
      <p class="eyebrow">Emergency-ready donor network</p>
      <h1>Blood Donor Finder</h1>
      <p class="header-subtitle">Register donors, find compatible blood groups, and manage institutions through a clean full-:
      <nav class="site-nav" style="display:flex;">
        <button data-section="home" onclick="showSection('home')">Home</button>
        <button data-section="register" onclick="showSection('register')">Register Donor</button>
        <button data-section="lists" onclick="showSection('lists')">Find Donor</button>
        <button data-section="about" onclick="showSection('about')">About</button>
        <button id="adminNav" data-section="admin" onclick="showSection('admin')>Admin Tools</button>
        <button data-section="ownerPanel" onclick="showSection('ownerPanel')>Owner Panel</button>
        <button id="logoutBtn" onclick="logout()" class="btn-secondary" style="display:none;">Logout</button>
      </nav>
    </div>
  </header>

```

SEARCH BLOOD DONOR FORM:

Purpose: To allow for patients to find a required blood

Fields:

- **Blood Donor Address(bd__ad):** A unique identifier for the donor.
 - ❖ **Input Type:** Text
 - ❖ **Validation:** Required, alphanumeric, unique
- **Blood Donor Location (bd__l):** The location of the donor.
 - ❖ **Input Type:** Text
 - ❖ **Validation:** Required, alphabetic

Design Considerations:

- **Clear Labels:** Ensure each field is clearly labeled.
- **Placeholder Text:** Provide examples in the placeholder text to guide the user.
- **Validation Messages:** Display error messages for invalid inputs.
- **Submit Button:** Clearly see the donor blood group .

GENERAL INPUT DESIGN:

- **Consistency:** Use a consistent design for all forms, ensuring that users can easily navigate and use the system.

- **Feedback:** Provide immediate feedback for user actions, such as form submissions and validation errors.
- **Accessibility:** Ensure forms are accessible to users with disabilities by following web accessibility guidelines.
- **Security:** Implement measures to prevent common security issues, such as SQL injection and cross-site scripting (XSS), through proper input validation and sanitization.
- **Responsiveness:** Design forms to be responsive, ensuring they work well on various screen sizes and devices.

3.7 OUTPUT DESIGN OF THE SYSTEM

Output design focuses on how the information generated by the system is presented to the users. Effective output design ensures that the system communicates information clearly, concisely, and in a user-friendly manner. Here's a detailed description of the output design for the Blood Donor Finder:

➤ WEB PAGES AND INTERFACES

1. Home Page

- **Description:** The **Home Page** is the main entry point of the Blood Donor Finder website. It provides users with quick access to essential features like searching for donors and registering as a donor..
- **Content:** The home page provides an overview of the system, including navigation, introduction, features, and quick access to donor search and registration.

2. Signup and Login Pages

- **Description:** The **Signup Page** allows new users (donors) to register in the system by providing their personal and contact details. This information is stored in the database and used to help others find donors during emergencies.
- **Content:** Forms for entering donor details (ID, name, address, contact), login credentials (ID and name), and submission buttons.

3. Donor Dashboard

- **Description:** The **Donor Dashboard** is a personalized interface for registered donors after login. It allows donors to **view, manage, and update their information**, and stay available for blood donation requests.

- **Content:** The donor dashboard provides a personalized interface where donors can view and update their profile, manage availability status, and respond to blood requests efficiently. Customer Dashboard.

4. Owner panel Pages

- **Description** The **Owner Panel (Admin Panel)** is a secure control system used by the website administrator to manage donors, monitor activities, and maintain the overall system efficiently
- **Content:** The Owner Panel is used by the administrator to manage donors, monitor blood requests, generate reports, and control overall system operations efficiently.

CHAPTER 4

METHODOLOGY

ITERATIVE METHODOLOGY

We followed the Iterative Methodology to develop our Blood Donor Finder website. In this approach, the system was built step by step in multiple iterations instead of developing everything at once. In the first iteration, we created the basic website structure and navigation. In the second iteration, we developed the donor registration module and connected it to the database. In the third iteration, we implemented the donor list display feature to allow users to view registered donors. Later, we integrated the QR code for quick access to the website and performed final testing and improvements. After each iteration, testing was done and necessary changes were made, which helped us improve the system gradually and ensure better performance and reliability.

4.1 STAGE OF THE ITERATIVE DEVELOPMENT PROCESS

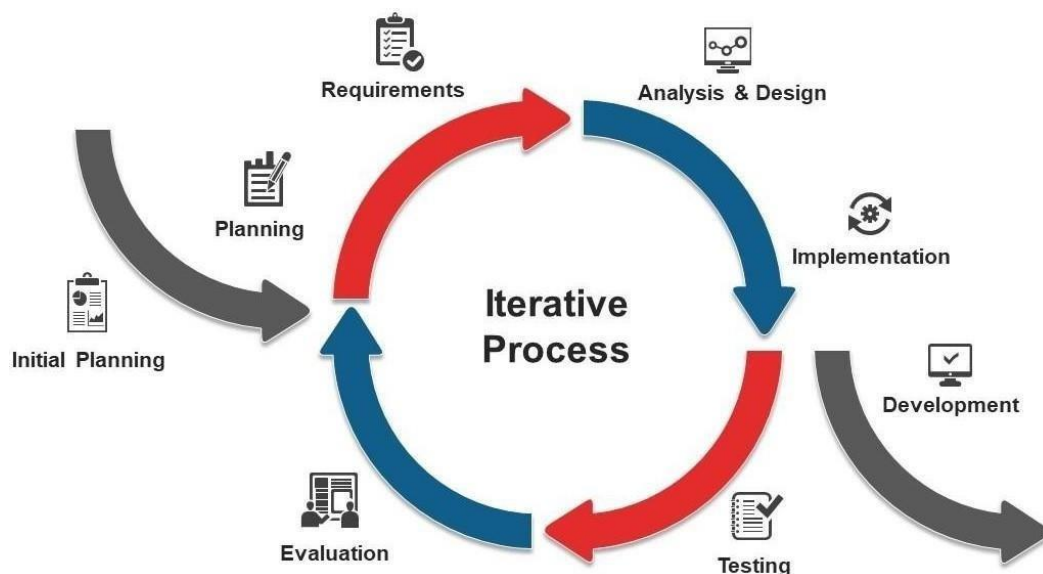


Figure 4.1 Iterative methodology

➤ Initial Planning:

- The project begins with an overall Initial Planning phase where you outline the goals and scope.
- For your blood donor finder, this means deciding you want a system that registers donors, lists them, and links to the website via QR codes.
-

➤ Planning:

- From initial planning, the Planning phase focuses on detailed preparation for the first iteration.

- You specify exactly what features to build first, such as registration forms, QR code scanning, and donar listing.

➤ **Requirements:**

- Here, you gather and define Requirements for the current iteration.
- For example, capturing donor name, blood group, phone number, and the ability to display donor lists.

➤ **Analysis & Design:**

- This phase analyzes requirements and designs the system architecture and UI.
- You decide how the website will look, how the database will store donor info, and how QR codes will link to the site.

➤ **Implementation:**

- This step involves coding and building the components designed.
- You develop the registration form, backend database, and generate the QR code.

➤ **Development:**

- Development here means assembling and integrating all features into a working product.
- The website is made functional with registration, donor lists, and contact features.

➤ **Testing:**

- Once developed, the system undergoes Testing to catch bugs and ensure requirements are met.
- You check that registrations save properly, donor lists display accurately, and QR code scanning works.

➤ **Evaluation:**

- After testing, the system is Evaluated — feedback is collected from users or testers.
- For example, users might suggest making the UI simpler or adding search filters.

4.2 ADVANTAGES OF BLOOD DONAR WEBSITE

1. Quick & Easy Access Through QR Code:

- Anyone can scan the QR code and instantly access the website.
- No need to search manually or install an app.
- Useful in emergencies where time is critical.

2. Saves Time During Emergencies:

- Patients or hospitals can quickly find donors.
- No need to search on social media or ask many people.
- Immediate access to donor list reduces delay.

3. Direct Contact With Donors:

- Users can see:
 - Donor Name
 - Blood Group
 - Phone Number
- They can directly call the donor without middle agents.
- Faster communication means faster blood availability.

4. Simple Registration Process:

- Donors can easily register by entering:
 - Name
 - Blood Group
 - Contact Number
- Encourages more people to participate.

5. Organized Donor Database:

- Blood groups are stored systematically.
- Easy to filter/search by blood group.
- Avoids confusion and duplicate effort.

6. Promotes Social Responsibility:

- Encourages people to donate blood.
- Builds a community of voluntary donors.
- Helps save lives.

4.3 DISADVANTAGES OF BLOOD DONAR WEBSITE

➤ Privacy & Data Security Issues:

Problem:

You are showing:

- Name
- Blood group
- Phone number

This is sensitive personal information.

Disadvantages:

- Misuse of phone numbers (spam calls, harassment)
- Data theft or hacking
- Identity misuse
- Privacy law violations

If the website is not secure, users may lose trust.

➤ Fake Registrations:

Problem:

Anyone can register with:

- Fake name
- Fake blood group
- Wrong phone number

Disadvantages:

- Wrong blood group information can risk lives
- People may not respond when called
- Reduces reliability of website

➤ No Medical Verification:

Your website only collects blood group information.

Disadvantage:

- Not all registered people are medically fit to donate
- Some may have:
 - Low hemoglobin\
 - Infections
 - Recent surgery
 - Age restrictions

Without verification, it can be unsafe.

➤ Emergency Response Delay

In emergency situations:

- Donor may not answer the phone
- Donor may be out of city
- Donor may not be available

Your system depends completely on manual calling.

➤ **Legal and Ethical Issues:**

Handling medical-related data may require:

- User consent
- Privacy policy
- Data protection rules

If not handled properly, it can cause legal problems.

➤ **Data Not Updated:**

People may:

- Change phone numbers
- Move to another city
- No longer be eligible to donate

If data is not updated regularly, it becomes useless.

➤ **No Location Filtering (If Not Implemented):**

If your website shows all donors without filtering:

- You may call someone far away
- Time is wasted in emergencies

➤ **Spam and Overuse of Donors:**

Same donors may:

- Get too many calls
- Be pressured to donate frequently

This can cause frustration and reduce participation.

CHAPTER 5

TESTING

5.1 INTRODUCTION OF TESTING

The Blood Donor Finder website was tested to ensure that all functionalities work properly and efficiently. First, the QR code was tested by scanning it using different mobile devices to verify that it correctly redirects users to the official website without any errors. The website loaded successfully under different network conditions, confirming that the QR redirection works properly. Next, the donor registration form was tested by entering valid details such as name, blood group, and phone number. The system successfully stored the data in the database and displayed a confirmation message. The registered donor details were then verified by opening the list option, where the newly added donor appeared correctly with accurate information.

Further testing was performed to validate input fields. The system was checked for empty fields, invalid phone numbers, and incorrect blood group entries to ensure that appropriate error messages were displayed. This confirmed that proper validation mechanisms are implemented. The donor list feature was also tested to ensure that all registered users are displayed correctly with their name, blood group, and contact number. The phone number link was tested by clicking on it to confirm that it opens the phone dialer, allowing users to contact donors easily.

Database testing was conducted to verify that all registered data is stored securely and retrieved accurately. Performance testing was done to check website loading speed and response time when multiple users access the system. Basic security testing was also performed to ensure that unauthorized users cannot modify or delete donor data. Overall, after conducting functional, validation, performance, and security testing, the system was found to be working successfully and efficiently. The Blood Donor Finder project meets its intended objective of connecting blood donors with people in need quickly and effectively.

5.2 TYPES OF TESTING

➤ Functional Testing:

Functional Testing is the process of verifying that all features of the Blood Donor Finder website work correctly according to the project requirements. In this testing, we check whether users can successfully register as blood donors by entering their name, selecting a valid blood group, and providing a correct phone number, and ensure that this information is properly stored in the database. It also includes testing whether the QR code correctly redirects users to the website, whether the registered donors are displayed properly in the list section, and whether users can view the donor's name, blood group, and contact number without errors.

➤ Validation Testing:

Validation Testing is the process of checking whether the user inputs entered on the Blood Donor Finder website are correct, complete, and in the proper format before being stored in the database. In this testing, we verify that required fields such as name, blood group, and phone number are not left empty and that the data entered follows the specified rules. For example, the name field should contain only alphabetic characters, the phone number should contain only digits and be of the correct length (such as 10 digits), and the blood group must be selected from valid options like A+, B+, O-, etc.

➤ **Database Testing:**

Database Testing is the process of verifying that all data in the Blood Donor Finder website is correctly stored, retrieved, updated, and deleted in the database without errors. In this testing, we check whether the information entered during registration such as name, blood group, and phone number is accurately saved in the database and properly displayed in the donor list section. It also ensures that duplicate records are not stored, deleted records are removed permanently, and data remains consistent even after refreshing the page or restarting the system.

➤ **UI/UX Testing (User Interface Testing):**

UI/UX Testing (User Interface Testing) is the process of evaluating whether the Blood Donor Finder website is visually clear, user-friendly, and easy to navigate. In this testing, we check that the registration form is simple to understand, buttons are clearly labeled and clickable, text is readable, and the layout is properly aligned on different screen sizes such as mobile phones, tablets, and computers. It also ensures that colors, fonts, and design elements are consistent and attractive, making the website comfortable to use.

➤ **Performance Testing:**

Performance Testing is the process of evaluating how well the Blood Donor Finder website performs under different conditions, especially when multiple users access it at the same time. In this testing, we check whether the website loads quickly after scanning the QR code, whether the registration and donor list pages open without delay, and whether data is stored and retrieved efficiently from the database. It also ensures that the system does not crash, slow down, or lose data when many users are registering or searching for donors simultaneously.

➤ **Security Testing:**

Security Testing is the process of ensuring that the Blood Donor Finder website is protected against unauthorized access, data breaches, and cyberattacks. In this testing, we verify that sensitive information such as users' names and phone numbers is securely stored and cannot be accessed or modified by unauthorized users. It includes checking for vulnerabilities like SQL injection, cross-site scripting (XSS), and other common security threats.

➤ **Compatibility Testing:**

Compatibility Testing is the process of verifying that the Blood Donor Finder website works properly across different devices, browsers, and operating systems. In this testing, we check whether the website functions correctly on various web browsers such as Chrome, Firefox, and Edge, and whether it displays properly on mobile phones, tablets, and desktop computers. It also ensures that the QR code redirects correctly on both Android and iPhone devices.

➤ **QR Code Testing:**

QR Code Testing is the process of ensuring that the QR code generated for the Blood Donor Finder website works correctly and reliably. In this testing, we check that scanning the QR code with different devices, such as Android phones and iPhones, opens the website without errors or delays. We also test the QR code from various sources, like printed paper, digital screens, and social media, to confirm it functions in all situations.

5.3 TESTING PERFORMED

➤ **Unit Testing:**

Individual components of the system were tested separately to ensure they function correctly.

- **Registration Module**

- Verified that users can register with valid name, phone number, and blood group.
- Checked validation for empty fields and incorrect inputs.

- **Search Module**

- Tested filtering donors based on blood group.
- Ensured correct donor details are displayed.

- **QR Code Module**

- Verified that scanning the QR code correctly opens the website.

➤ **Integration Testing:**

Tested how different modules work together.

- Confirmed that data entered during registration is stored in the database and retrieved correctly during search.
- Ensured smooth navigation between registration and emergency request pages.

➤ **System Testing:**

The complete system was tested as a whole.

- **End-to-end testing:**

- User scans QR code → opens website → registers → data stored → another user searches → donor details displayed.
- Verified system performance under normal usage conditions.

➤ **User Interface Testing:**

Tested the usability and responsiveness of the website.

- Checked layout on mobile and desktop devices.
- Ensured buttons, forms, and navigation links work properly.
- Verified readability and user-friendly design.

➤ **Validation Testing:**

Ensured proper input validation.

- Phone number must be numeric and of valid length.
- Blood group must be selected from predefined options.
- Required fields cannot be left empty.

➤ **Performance Testing:**

- Tested loading time of the website.
- Ensured quick response when searching for donors.
- Verified system handles multiple users efficiently (basic level).

➤ **Security Testing:**

- Ensured user data is stored securely.
- Verified that unauthorized access to donor data is restricted.
- Checked basic protection against invalid input

CHAPTER 6

IMPLEMENTATION

IMPLEMENTATION

The Blood Donor Finder system is implemented as a web-based application designed to connect blood donors with recipients efficiently. The system is developed using front-end technologies such as HTML, CSS, and JavaScript to create a user-friendly interface, while the back-end is implemented using PHP, which handles server-side processing and communication with the MySQL database. When a user accesses the website (via URL or QR code), the homepage provides options for donor registration, blood search, and request submission. New users can register by filling out a form that collects essential details such as name, age, gender, blood group, contact number, and location. This data is sent to the server and stored in the database using SQL queries.

The database is designed using multiple tables such as donor, blood request, registration, and donation, ensuring proper organization and normalization of data. Each donor is assigned a unique ID, which is used to maintain relationships between tables through foreign keys. When a user searches for blood, the system processes the input (such as blood group and city) and retrieves matching donor records from the database using SELECT queries.

6.1 TABLES USED

DATABASE: BLOOD DONOR FINDER

The **Blood Donor Finder Database** is used to store and manage all the information related to donors, users, and blood requests. It ensures fast retrieval of data and helps in efficiently connecting donors with recipients' identification.

```
CREATE DATABASE blood_donor_finder;  
USE blood_donor_finder;
```

➤ DONOR TABLE

The **Donor Table** is the main table in the system that stores all the details of registered blood donors. It helps in searching and retrieving donor information based on blood group and location during emergencies.

```
CREATE TABLE donor (  
    donor_id INT PRIMARY KEY AUTO_INCREMENT,  
    full_name VARCHAR(100) NOT NULL,  
    age INT,  
    gender VARCHAR(10),  
    blood_group VARCHAR(5),  
    phone VARCHAR(15) UNIQUE,  
    email VARCHAR(100),  
    city VARCHAR(50),  
    last_donation_date DATE  
);
```

➤ REGISTRATION TABLE

The **Registration Table** stores information about users (donors) when they sign up on the website. It acts as a link between the user account and donor details, helping manage registration records efficiently.

```
CREATE TABLE registration (  
    reg_id INT PRIMARY KEY AUTO_INCREMENT,  
    donor_id INT,  
    registration_date DATE,  
    FOREIGN KEY (donor_id) REFERENCES donor(donor_id)  
)
```

➤ REQUEST TABLE

The **Request Table** stores all the blood requests made by users or hospitals. It contains details about the patient, required blood group, location, and request status. This table helps connect people in need with available donors quickly

```
CREATE TABLE blood_request (  
    request_id INT PRIMARY KEY AUTO_INCREMENT,  
    patient_name VARCHAR(100),  
    blood_group VARCHAR(5),  
    units_required INT,  
    hospital_name VARCHAR(100),  
    city VARCHAR(50),  
    contact_number VARCHAR(15),  
    request_date DATE);
```

➤ DONATION TABLE

The **Donation Table** stores information about actual blood donation activities performed by donors. It records when a donor donates blood, where the donation happened, and links it with donor and request details.

```
CREATE TABLE donation (  
    donation_id INT PRIMARY KEY AUTO_INCREMENT,  
    donor_id INT,  
    donation_date DATE,  
    hospital_name VARCHAR(100),  
    FOREIGN KEY (donor_id) REFERENCES donor(donor_id)  
);
```

➤ SAMPLE DATA

Sample Data refers to a set of example records inserted into the database tables to test and demonstrate the working of the system. It helps in verifying whether the system is functioning correctly and producing expected outputs.

```
INSERT INTO donor (full_name, age, gender, blood_group, phone, email, city)
VALUES
('Rahul Kumar', 22, 'Male', 'O+', '9876543210', 'rahul@gmail.com', 'Bangalore'),
('Anjali Sharma', 24, 'Female', 'A+', '9123456780', 'anjali@gmail.com', 'Delhi');
```

6.2 DATABASE CONNECTION

Database Connection is the process of connecting the website (frontend/backend) with the database (MySQL/SQLite) to store, retrieve, update, and delete data.

```
const connectionString =
  process.env.DATABASE_URL || 'postgres://postgres:password@localhost:5432/blood_donation';

const pool = new Pool({ connectionString });

console.log('Initializing PostgreSQL database and seeding admins...\n');

async function setup() {
  try {
    await pool.query(`
      CREATE TABLE IF NOT EXISTS users (
        id SERIAL PRIMARY KEY,
        google_id TEXT UNIQUE,
        email TEXT UNIQUE,
        phone TEXT UNIQUE,
        name TEXT,
        is_admin BOOLEAN DEFAULT FALSE
      )
    `);
    console.log('OK Users table created/exists');

    await pool.query(`
      CREATE TABLE IF NOT EXISTS admins (
        id SERIAL PRIMARY KEY,
        username TEXT UNIQUE,
        password_hash TEXT
      )
    `);
    console.log('OK Admins table created/exists');

    await pool.query(`
      CREATE TABLE IF NOT EXISTS institutions (
        id SERIAL PRIMARY KEY,
        name TEXT UNIQUE
      )
    `);
  }
}
```

6.3 INSERT OPERATION

The **INSERT operation** is used to add new records into a database table. In the **Blood Donor Finder Website**, this operation is mainly used when:

- A new donor registers
- A user creates a blood request
- A new user account is created

```

app.get('/api/admin/institutions-with-donors', requireAdmin, async (req, res) => {
  try {
    const query = `
      SELECT
        i.id AS institution_id,
        i.name AS institution_name,
        d.id AS donor_id,
        d.name,
        d.age,
        d.blood_group,
        d.contact,
        d.address
      FROM institutions i
      LEFT JOIN donors d ON d.institution_id = i.id
      ORDER BY LOWER(i.name), LOWER(COALESCE(d.name, ''))
    `;
    const { rows } = await pool.query(query);

    const institutions = [];
    const byId = new Map();

    for (const row of rows) {
      if (!byId.has(row.institution_id)) {
        const institution = {
          id: row.institution_id,
          name: row.institution_name,
          donors: []
        };
        byId.set(row.institution_id, institution);
        institutions.push(institution);
      }
    }
  }
}

```

6.4 UPDATE OPERATION

The **UPDATE operation with database connection** is used to modify existing records in the database through backend programming (like PHP or Node.js). It allows users to edit previously stored data such as donor details, availability status, or profile information.

```

app.get('/api/admin/institutions-with-donors', requireAdmin, async (req, res) => {
  try {
    const query = `
      SELECT
        i.id AS institution_id,
        i.name AS institution_name,
        d.id AS donor_id,
        d.name,
        d.age,
        d.blood_group,
        d.contact,
        d.address
      FROM institutions i
      LEFT JOIN donors d ON d.institution_id = i.id
      ORDER BY LOWER(i.name), LOWER(COALESCE(d.name, ''))
    `;
    const { rows } = await pool.query(query);

    const institutions = [];
    const byId = new Map();

    for (const row of rows) {
      if (!byId.has(row.institution_id)) {
        const institution = {
          id: row.institution_id,
          name: row.institution_name,
          donors: []
        };
        byId.set(row.institution_id, institution);
        institutions.push(institution);
      }
    }
  }
}

```

6.5 DELETION OPERATION

The **DELETE operation with database connection** is used to remove existing records from the database using backend programming (such as PHP or Node.js). It helps in maintaining clean and accurate data by deleting unwanted, duplicate, or invalid records.

```
app.delete('/api/donors/:id', requireAdmin, async (req, res) => {
  try {
    const result = await pool.query('DELETE FROM donors WHERE id = $1', [req.params.id]);
    if (result.rowCount === 0) {
      return res.status(404).json({ error: 'Not found' });
    }
    res.json({ message: 'Deleted donor' });
  } catch (err) {
    res.status(500).json({ error: 'Database error' });
  }
});

// Delete Institution
app.delete('/api/institutions/:institution', requireAdmin, async (req, res) => {
  const result = await pool.query('DELETE FROM institutions WHERE name = $1', [req.params.institution]);
  if (result.rowCount === 0) return res.status(404).json({ error: 'Not found' });
  res.json({ message: 'Deleted' });
});

app.listen(PORT, () => console.log(`Server running on http://localhost:${PORT}`));
```

CHAPTER 7

SNAPSHOTS

SNAPSHOTS

This section shows snapshots of the system's interface, providing a visual representation of its working. The images provide a glimpse at the system's design, navigation, and important features, highlighting its usability and performance. The images below help readers understand the system's capabilities and potential benefits.

7.1 LOGIN PAGE

The Login Page of the Blood Donor Finder 2.0 platform serves as the primary security gateway, ensuring that only authorized donors, seekers, and administrators can access their respective dashboards. The interface is designed with a clean, minimalist aesthetic using Tailwind CSS, featuring a centralized login card that includes input fields for unique identifiers such as email or username and a secure password. To enhance user experience, the page incorporates real-time form validation to detect empty fields or incorrect formats before the request is sent to the server.

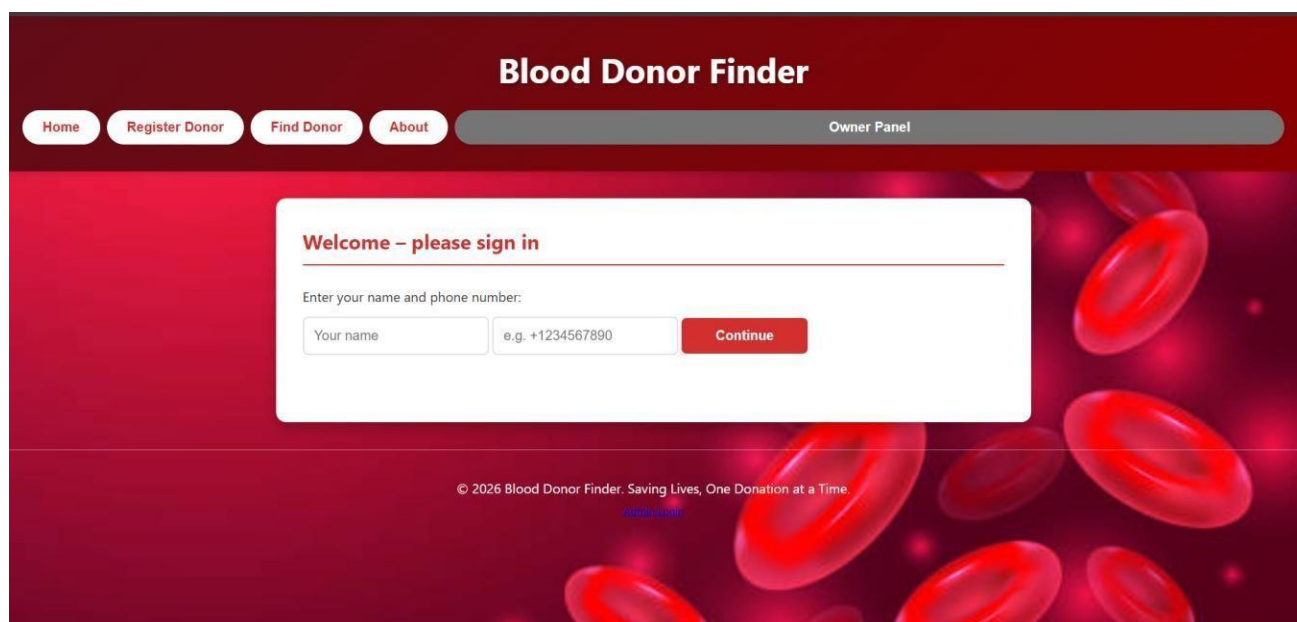


Figure 7.1 Login Page

7.2 HOME PAGE

The home page serves as the system's centre, providing users with a navigational starting point as well as a summary of key information or features. It provides a userfriendly interface that allows them to quickly access important functionality and manage the system.

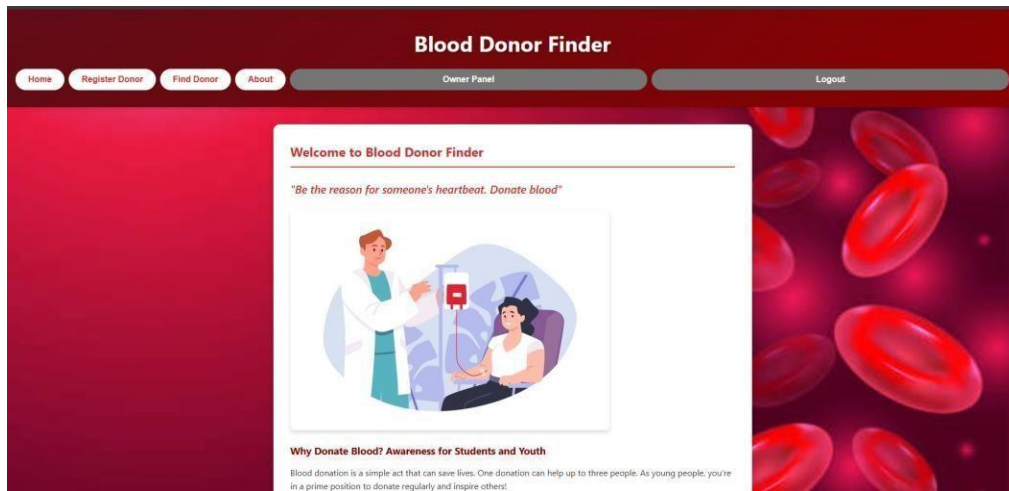


Figure 7.2 Home page

7.3 DONOR REGISTER PAGE

The **Donor Register Page** is an essential component of the Blood Donor Finder website that allows new users to sign up as blood donors.

The screenshot shows the 'Register as a Donor' form on the 'Blood Donor Finder' website. The header and navigation bar are the same as in the home page. The form is a white box with a red border. It contains the following fields: 'Institution:' with a text input field (example: 'e.g., City University'), 'Full Name:' with a text input field (placeholder: 'Enter your full name'), 'Age:' with a text input field (placeholder: 'Enter your age'), 'Blood Group:' with a dropdown menu (placeholder: 'Select Blood Group'), 'Contact (Phone):' with a text input field (placeholder: 'Enter phone number'), and 'Address:' with a text input field (placeholder: 'Enter your address'). At the bottom of the form are two buttons: 'Register' (red) and 'Clear Form' (grey).

Figure 7.3 Donor Register Page

7.4 DONOR FINDER PAGE

The **Donor Finder Page** is a key feature of the Blood Donor Finder system that allows users to search and locate available blood donors based on specific requirements. This page is designed to provide quick and efficient access to donor information during emergency situations.

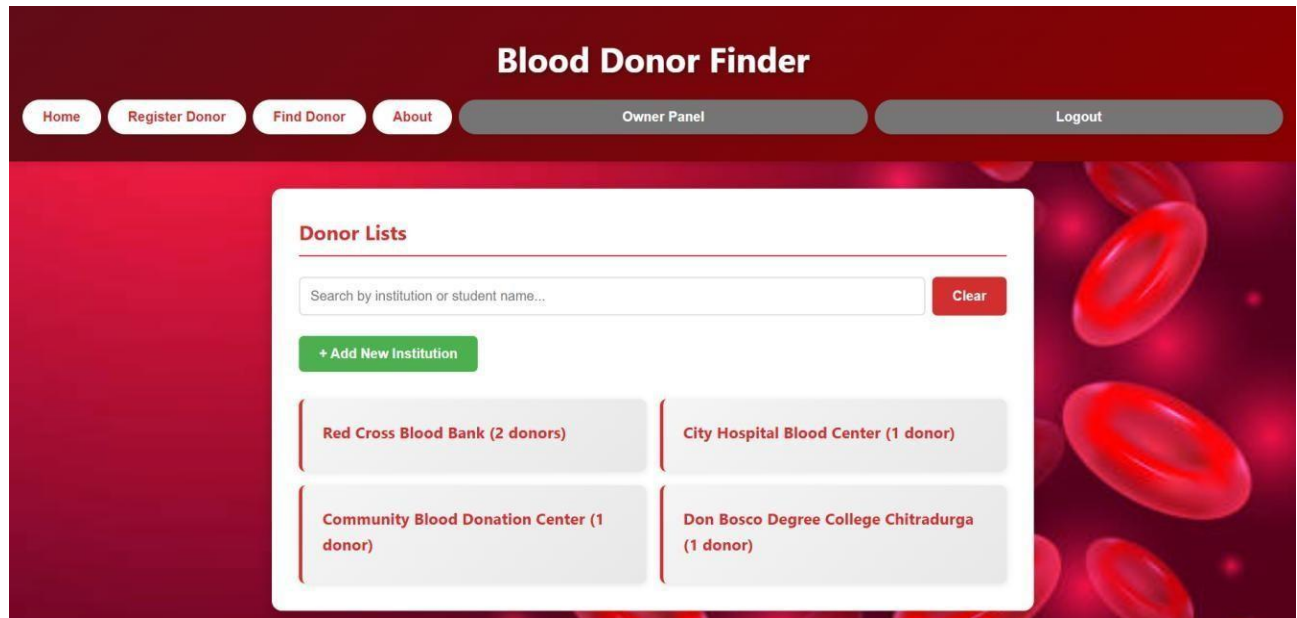


Figure 7.4 Donor Finder Page

7.5 OWNER/ADMIN PANEL

The **Owner/Admin Panel** is a secure and centralized control interface of the Blood Donor Finder system. It is designed for the administrator (website owner) to manage all operations related to donors, institutions, and system data.

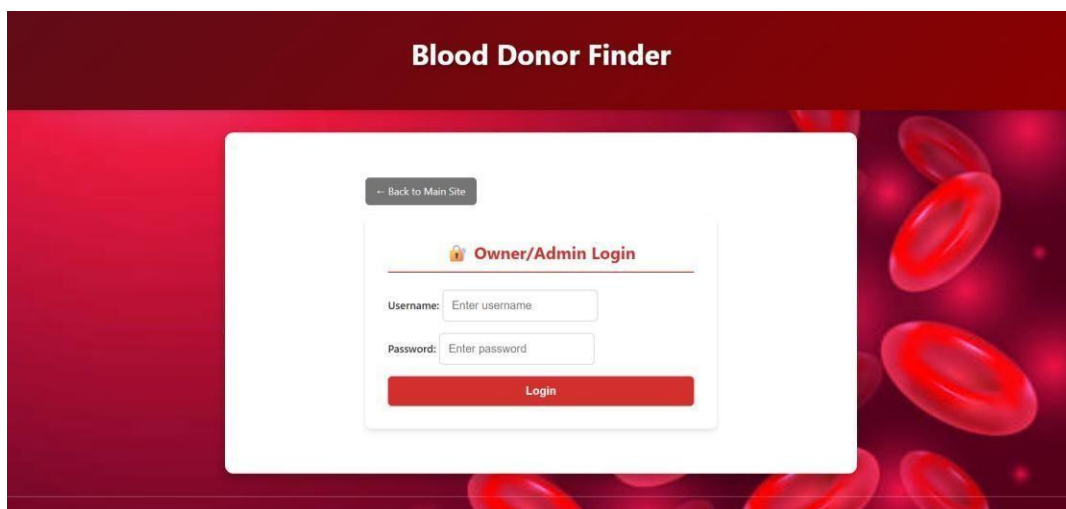


Figure 7.5 Owner/Admin Panel Page

7.6 ABOUT PAGE

The page begins with an **introduction section**, explaining that the Blood Donor Finder is a purpose-driven platform designed to connect voluntary blood donors with individuals and medical institutions in need. It highlights the common problem of difficulty in finding the right donor at the right time, especially during emergencies.

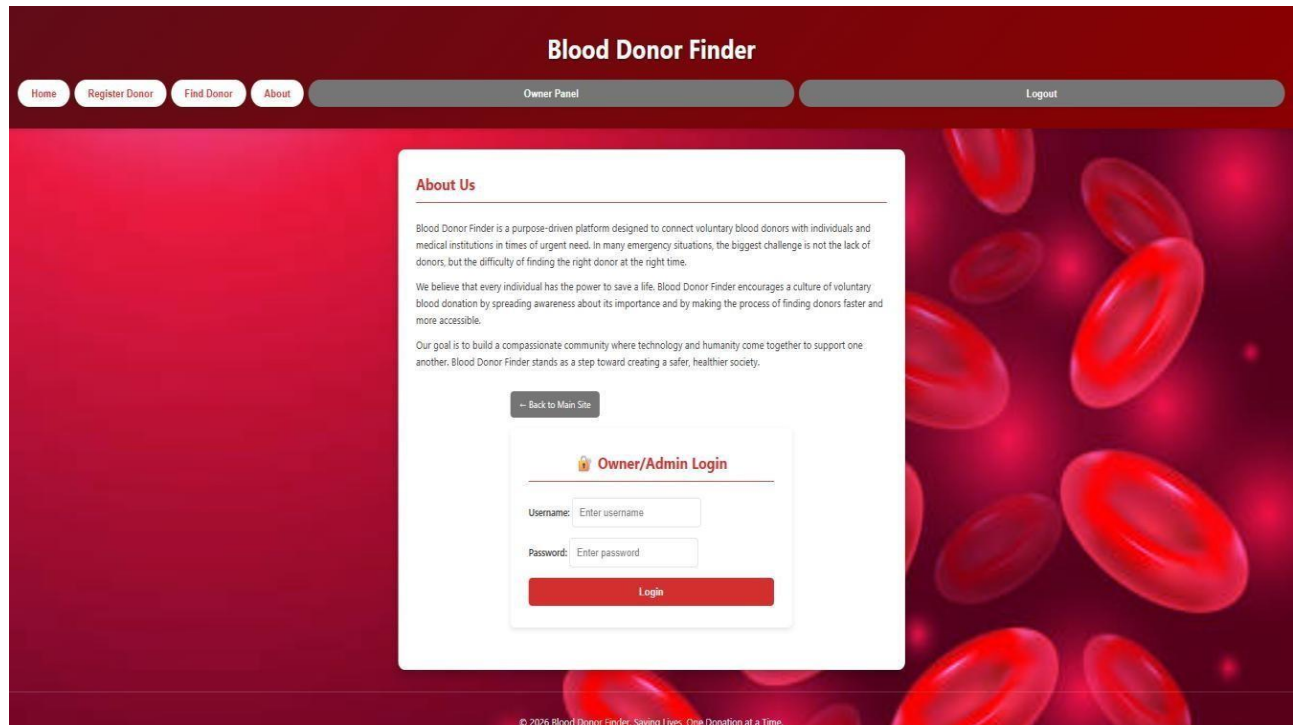


Figure 7.6 About Page

CHAPTER 8

CONCLUSION

8.1 CONCLUSION

In conclusion, the development of the web-based Blood Donor Finder 2.0 system using Flask and MySQL marks a significant step towards modernizing emergency healthcare coordination. This project aimed to provide donors and seekers with a seamless platform for managing donor registrations, blood requests, and real-time availability tracking. Key features such as robust database design, secure user authentication mechanisms, and intuitive web interfaces were integral to achieving these goals.

Throughout the project, meticulous attention was given to database design using MySQL. The schema was structured to efficiently store and manage crucial data entities such as donor profiles, blood groups, hospital locations, and donation history. Utilizing foreign key constraints ensured data integrity, maintaining relational consistency across tables. This foundational design facilitated streamlined operations and enabled secure data management, which is crucial for a system handling sensitive personal health information.

The implementation of Flask as the web framework provided a flexible and scalable architecture for the donor management system. Flask routes were meticulously crafted to handle various functionalities, including donor and seeker authentication, blood request management, location-based searching, and administrative oversight. Session management was employed to handle user states effectively, ensuring secure and personalized interactions for every user on the platform.

8.2 FEATURE WORKS

Looking ahead, several areas for future enhancement and development have been identified to further elevate the Blood Donor Finder 2.0 system. Implementing advanced search and filtering functionalities will empower users to find specific blood types based on precise geographical proximity or urgency levels, enhancing the overall life-saving potential of the platform.

Real-time updates using WebSocket technology represent another pivotal area for improvement. Enabling real-time notifications for donors and seekers regarding urgent blood requirements, request approvals, or nearby donation drives can significantly enhance communication and user engagement. Furthermore, integrating SMS gateways to facilitate instant alerts for emergency matches will streamline the connection process, offering users a seamless and efficient experience directly from their mobile devices.

Analytics and reporting capabilities are also essential for future iterations of the system. By implementing robust analytics tools, administrators can gain valuable insights into blood demand trends, regional shortages, and donor participation rates. These insights will empower healthcare organizations to make data-driven decisions, optimize blood drive scheduling, and tailor community outreach strategies to meet patient needs effectively.

CHAPTER 9

REFERENCE

REFERENCE

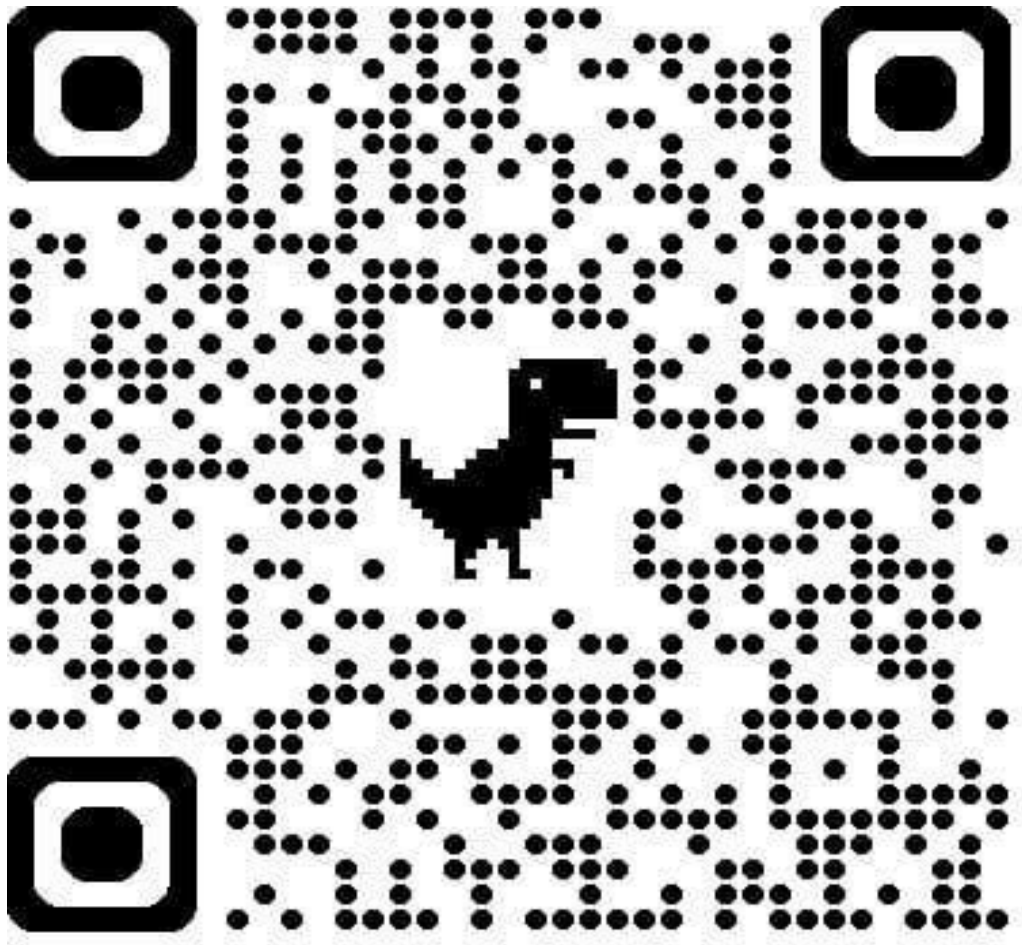
BOOKS:

1. Eloquent JavaScript: A Modern Introduction to Programming By Marijn Haverbeke
2. Node.js Design Patterns: Design and implement production-grade Node.js applications By Mario Casciaro and Luciano Mammino
3. SQLite Database System: Design and Implementation By Sibsankar Haldar and Malaya Kumar Nayak

WEB SITE:

- ❖ <https://developer.mozilla.org/en-US/docs/Web/JavaScript> (Official JavaScript Documentation)
- ❖ <https://nodejs.org/en/docs/> (Node.js Runtime Documentation)
- ❖ <https://www.sqlite.org/docs.html> (MySQLite/SQLite Database Reference)
- ❖ <https://www.w3schools.com/js/default.asp> (JavaScript Web Tutorials)
- ❖ <https://www.geeksforgeeks.org/blood-bank-management-system-using-node-js/> (Project Reference)
- ❖ <https://gemini.google.com/> (AI Research and Content Assistance)

BLOOD DONAR FINDER



<https://mithun-mithu.github.io/Blood-Donor-Finder---2.0/>